pandas.DataFrame.append

`DataFrame.append(other, ignore_index=False, verify_integrity=False, sort=False)` [source]

Append rows of *other* to the end of caller, returning a new object.

> **Deprecated since version 1.4.0:** Use `concat()` instead. For further details see Deprecated DataFrame.append and Series.append

Columns in *other* that are not in the caller are added as new columns.

Parameters:

other : *DataFrame or Series/dict-like object, or list of these*

The data to append.

ignore_index : *bool, default False*

If True, the resulting axis will be labeled 0, 1, …, n - 1.

verify_integrity : *bool, default False*

If True, raise ValueError on creating index with duplicates.

sort : *bool, default False*

Sort columns if the columns of *self* and *other* are not aligned.

> **Changed in version 1.0.0:** Changed to not sort by default.

Returns:

DataFrame

A new DataFrame consisting of the rows of caller and the rows of *other*.

See also

`concat`

General function to concatenate DataFrame or Series objects.

Notes

If a list of dict/series is passed and the keys are all contained in the DataFrame's index, the order of the columns in the resulting DataFrame will be unchanged.

Iteratively appending rows to a DataFrame can be more computationally intensive than a single concatenate. A better solution is to append those rows to a list and then concatenate the list with the original DataFrame all at once.

Examples

```
>>> df = pd.DataFrame([[1, 2], [3, 4]], columns=list('AB'), index=['x', 'y'])
>>> df
   A  B
x  1  2
y  3  4
>>> df2 = pd.DataFrame([[5, 6], [7, 8]], columns=list('AB'), index=['x', 'y'])
>>> df.append(df2)
   A  B
x  1  2
y  3  4
x  5  6
y  7  8
```

With *ignore_index* set to True:

```
>>> df.append(df2, ignore_index=True)
   A  B
0  1  2
1  3  4
2  5  6
3  7  8
```

pandas.Series.iteritems

Series.iteritems() [source]

Lazily iterate over (index, value) tuples.

> ***Deprecated since version 1.5.0:*** iteritems is deprecated and will be removed in a future version. Use .items instead.

This method returns an iterable tuple (index, value). This is convenient if you want to create a lazy iterator.

Returns: **iterable**

Iterable of tuples containing the (index, value) pairs from a Series.

See also

`Series.items`
Recommended alternative.

`DataFrame.items`
Iterate over (column name, Series) pairs.

`DataFrame.iterrows`
Iterate over DataFrame rows as (index, Series) pairs.

Previous
pandas.Series.items

Next
pandas.Series.keys

© 2023 pandas via NumFOCUS, Inc. Hosted by OVHcloud.
Created using Sphinx 4.5.0.

What's new in 1.0.0 (January 29, 2020)

These are the changes in pandas 1.0.0. See Release notes for a full changelog including other versions of pandas.

> **Note**
>
> The pandas 1.0 release removed a lot of functionality that was deprecated in previous releases (see below for an overview). It is recommended to first upgrade to pandas 0.25 and to ensure your code is working without warnings, before upgrading to pandas 1.0.

New deprecation policy

Starting with pandas 1.0.0, pandas will adopt a variant of SemVer to version releases. Briefly,

- Deprecations will be introduced in minor releases (e.g. 1.1.0, 1.2.0, 2.1.0, ...)
- Deprecations will be enforced in major releases (e.g. 1.0.0, 2.0.0, 3.0.0, ...)
- API-breaking changes will be made only in major releases (except for experimental features)

See Version policy for more.

Enhancements

Using Numba in `rolling.apply` and `expanding.apply`

We've added an `engine` keyword to `apply()` and `apply()` that allows the user to execute the routine using Numba instead of Cython. Using the Numba engine can yield significant performance gains if the apply function can operate on numpy arrays and the data set is larger (1 million rows or greater). For more details, see rolling apply documentation (GH28987, GH30936)

Defining custom windows for rolling operations

We've added a `pandas.api.indexers.BaseIndexer()` class that allows users to define how window bounds are created during `rolling` operations. Users can define their own `get_window_bounds` method on a `pandas.api.indexers.BaseIndexer()` subclass that will generate the start and end indices used for each window during the rolling aggregation. For more details and example usage, see the custom window rolling documentation

Converting to markdown

We've added `to_markdown()` for creating a markdown table (GH11052)

```
In [1]: df = pd.DataFrame({"A": [1, 2, 3], "B": [1, 2, 3]}, index=['a', 'a', 'b'])

In [2]: print(df.to_markdown())
|    |   A |   B |
|:---|----:|----:|
| a  |   1 |   1 |
| a  |   2 |   2 |
| b  |   3 |   3 |
```

Experimental new features

Experimental `NA` scalar to denote missing values

A new `pd.NA` value (singleton) is introduced to represent scalar missing values. Up to now, pandas used several values to represent missing data: `np.nan` is used for this for float data, `np.nan` or `None` for object-dtype data and `pd.NaT` for datetime-like data. The goal of `pd.NA` is to provide a "missing" indicator that can be used consistently across data types.

`arrays.IntegerArray` comparisons return `arrays.BooleanArray`

Comparison operations on a `arrays.IntegerArray` now returns a `arrays.BooleanArray` rather than a NumPy array (GH29964).

pandas 0.25.x

```
In [1]: a = pd.array([1, 2, None], dtype="Int64")
In [2]: a
Out[2]:
<IntegerArray>
[1, 2, NaN]
Length: 3, dtype: Int64

In [3]: a > 1
Out[3]:
array([False,  True, False])
```

pandas 1.0.0

```
In [45]: a = pd.array([1, 2, None], dtype="Int64")

In [46]: a > 1
Out[46]:
<BooleanArray>
[False, True, <NA>]
Length: 3, dtype: boolean
```

Note that missing values now propagate, rather than always comparing unequal like `numpy.nan`. See Experimental NA scalar to denote missing values for more.

By default `Categorical.min()` now returns the minimum instead of np.nan

When `Categorical` contains `np.nan`, `Categorical.min()` no longer return `np.nan` by default (skipna=True) (GH25303)

pandas 0.25.x

```
In [1]: pd.Categorical([1, 2, np.nan], ordered=True).min()
Out[1]: nan
```

pandas 1.0.0

```
In [47]: pd.Categorical([1, 2, np.nan], ordered=True).min()
Out[47]: 1
```

Default dtype of empty `pandas.Series`

Initialising an empty `pandas.Series` without specifying a dtype will raise a `DeprecationWarning` now (GH17261). The default dtype will change from `float64` to `object` in future releases so that it is consistent with the behaviour of `DataFrame` and `Index`.

pandas 1.0.0

```
In [1]: pd.Series()
Out[2]:
DeprecationWarning: The default dtype for empty Series will be 'object' instead of 'float64' in a future
version. Specify a dtype explicitly to silence this warning.
Series([], dtype: float64)
```

pandas.DataFrame.merge

DataFrame.merge(*right, how='inner', on=None, left_on=None, right_on=None, left_index=False, right_index=False, sort=False, suffixes=('_x', '_y'), copy=True, indicator=False, validate=None*) [source]

Merge DataFrame or named Series objects with a database-style join.

A named Series object is treated as a DataFrame with a single named column.

The join is done on columns or indexes. If joining columns on columns, the DataFrame indexes *will be ignored*. Otherwise if joining indexes on indexes or indexes on a column or columns, the index will be passed on. When performing a cross merge, no column specifications to merge on are allowed.

> **Warning**
>
> If both key columns contain rows where the key is a null value, those rows will be matched against each other. This is different from usual SQL join behaviour and can lead to unexpected results.

Parameters:

right : *DataFrame or named Series*

Object to merge with.

how : *{'left', 'right', 'outer', 'inner', 'cross'}, default 'inner'*

Type of merge to be performed.

- left: use only keys from left frame, similar to a SQL left outer join; preserve key order.
- right: use only keys from right frame, similar to a SQL right outer join; preserve key order.
- outer: use union of keys from both frames, similar to a SQL full outer join; sort keys lexicographically.
- inner: use intersection of keys from both frames, similar to a SQL inner join; preserve the order of the left keys.
- cross: creates the cartesian product from both frames, preserves the order of the left keys.

> *New in version 1.2.0.*

on : *label or list*

Column or index level names to join on. These must be found in both DataFrames. If *on* is None and not merging on indexes then this defaults to the intersection of the columns in both DataFrames.

left_on : *label or list, or array-like*

Column or index level names to join on in the left DataFrame. Can also be an array or list of arrays of the length of the left DataFrame. These arrays are treated as if they are columns.

right_on : *label or list, or array-like*

Column or index level names to join on in the right DataFrame. Can also be an array or list of arrays of the length of the right DataFrame. These arrays are treated as if they are columns.

left_index : *bool, default False*

Use the index from the left DataFrame as the join key(s). If it is a MultiIndex, the number of keys in the other DataFrame (either the index or a number of columns) must match the number of levels.

right_index : *bool, default False*

Use the index from the right DataFrame as the join key. Same caveats as left_index.

sort : *bool, default False*

Sort the join keys lexicographically in the result DataFrame. If False, the order of the join keys depends on the join type (how keyword).

suffixes : *list-like, default is ("_x", "_y")*

A length-2 sequence where each element is optionally a string indicating the suffix to add to overlapping column names in *left* and *right* respectively. Pass a value of *None* instead of a string to indicate that the column name from *left* or *right* should be left as-is, with no suffix. At least one of